

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



Build a Scrollable List

Oleh:

Muhammad Guntur Ricky Adhitya NIM. 2410817310003

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Guntur Ricky Adhitya

NIM : 2410817310003

Menyetujui,
Asisten Praktikum

Menyetujui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom., M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI.....	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL.....	5
SOAL 1.....	6
A. Source Code.....	7
B. Output Program.....	69
C. Pembahasan.....	79
SOAL 2.....	83
A. Pembahasan.....	83
SOAL 3.....	84
A. Pembahasan.....	84
LINK GITHUB.....	85

DAFTAR GAMBAR

Gambar 1. Home Page Compose English Modul 3.....	69
Gambar 2. Detail Page Compose English Modul 3.....	70
Gambar 3. Language Page Compose English Modul 3.....	71
Gambar 4. Home Page Compose Indonesia Modul 3.....	72
Gambar 5. Detail Page Compose Indonesia Modul 3.....	73
Gambar 6. Home Page XML English Modul 3.....	74
Gambar 7. Detail Page XML English Modul 3.....	75
Gambar 8. Language Page XML English Modul 3.....	76
Gambar 9. Home Page XML Indonesia Modul 3.....	77
Gambar 10. Detail Page XML Indonesia Modul 3.....	78

DAFTAR TABEL

Tabel 1. CodeforcesProblem Modul 3.....	7
Tabel 2. ProblemRepository Modul 3.....	7
Tabel 3. DetailState Modul 3.....	9
Tabel 4. DetailViewModel Modul 3.....	9
Tabel 5. HomeState Modul 3.....	11
Tabel 6. HomeViewModel Modul 3.....	11
Tabel 7. LanguageModel Modul 3.....	12
Tabel 8. LanguageState Modul 3.....	12
Tabel 9. LanguageViewModel Modul 3.....	13
Tabel 10. DetailScreen Modul 3.....	14
Tabel 11. HomeScreen Modul 3.....	18
Tabel 12. LanguageScreen Modul 3.....	24
Tabel 13. AppNavigation Modul 3.....	27
Tabel 14. RoutingNames Modul 3.....	28
Tabel 15. MainActivity Compose Modul 3.....	29
Tabel 16. DetailFragment Modul 3.....	30
Tabel 17. ProblemListAdapter Modul 3.....	32
Tabel 18. CarouselAdapter Modul 3.....	34
Tabel 19. HomeFragment Modul 3.....	35
Tabel 20. LanguageAdapter Modul 3.....	39
Tabel 21. LanguageFragment Modul 3.....	41
Tabel 22. DiffCallback Modul 3.....	43
Tabel 23. MainActivity XML Modul 3.....	44
Tabel 24. MainActivity XML Modul 3.....	45
Tabel 25. FragmentDetail XML Modul 3.....	45
Tabel 26. FragmentHome Modul 3.....	49
Tabel 27. FragmentLanguage XML Modul 3.....	50
Tabel 28. ItemLanguage XML Modul 3.....	51
Tabel 29. ItemProblemCard XML Modul 3.....	52
Tabel 30. ItemProblemCarousel XML Modul 3.....	55
Tabel 31. HomeMenu XML Modul 3.....	55
Tabel 32. NavGraph XML Modul 3.....	56

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian bahasa
11. Gunakan Icon dari library 'material' (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah "Item Highlight" yang menggunakan Carousel atau LazyRow dengan ketentuan:
 - Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list

A. Source Code

Generic Source Code / Module

Tabel 1. CodeforcesProblem Modul 3

```
package com.mobil.modul3{xml/compose}.data

import androidx.annotation.DrawableRes
import androidx.annotation.RawRes
import androidx.annotation.StringRes

data class CodeforcesProblem(
    val problemId: String,
    @StringRes val title: Int,
    @StringRes val description: Int,
    val url: String,
    @RawRes val solutionCode: Int,
    @DrawableRes val img: Int,
)
```

Tabel 2. ProblemRepository Modul 3

```
package com.mobil.modul3{xml/compose}.data

import com.mobil.modul3{xml/compose}.R

object ProblemRepository {
    private val problemList = listOf(
        CodeforcesProblem(
```

```

        problemId = "2220A",

        title = R.string.problem_2220a_title,

        description = R.string.problem_2220a_desc,

                                url      =
"https://codeforces.com/contest/2220/problem/A",

        solutionCode = R.raw.problem_2220a_code, //
Updated

        img = R.drawable.a_blocked

    ),

    CodeforcesProblem(

        problemId = "2209B",

        title = R.string.problem_2209b_title,

        description = R.string.problem_2209b_desc,

                                url      =
"https://codeforces.com/contest/2209/problem/B",

        solutionCode = R.raw.problem_2209b_code, //
Updated

        img = R.drawable.b_array_operation

    ),

    CodeforcesProblem(

        problemId = "2209A",

        title = R.string.problem_2209a_title,

        description = R.string.problem_2209a_desc,

                                url      =
"https://codeforces.com/contest/2209/problem/A",

        solutionCode = R.raw.problem_2209a_code, //
Updated

        img = R.drawable.a_initial_config

    ),

```



```

        CodeforcesProblem(
            problemId = "2125B",
            title = R.string.problem_2125b_title,
            description = R.string.problem_2125b_desc,
                                                    url      =
"https://codeforces.com/contest/2125/problem/B",
            solutionCode = R.raw.problem_2125b_code, //
Updated
            img = R.drawable.b_left_and_down
        ),
        CodeforcesProblem(
            problemId = "2209C",
            title = R.string.problem_2209c_title,
            description = R.string.problem_2209c_desc,
                                                    url      =
"https://codeforces.com/contest/2209/problem/C",
            solutionCode = R.raw.problem_2209c_code, //
Updated
            img = R.drawable.c_find_the_zero
        )
    )

    fun getProblemById(id: String): CodeforcesProblem? {
        return problemList.find { it.problemId == id }
    }

    fun getAllProblems(): List<CodeforcesProblem> =
problemList
}

```

Tabel 3. DetailState Modul 3

```
package
com.mobil.modul3{xml/compose}.ui.{fragments/screens}.detail

data class DetailState(
    val titleRes: Int = 0,
    val descRes: Int = 0,
    val imgRes: Int = 0,
    val code: String = "",
    val isNotFound: Boolean = false
)
```

Tabel 4. DetailViewModel Modul 3

```
package com.mobil.modul3xml.ui.{fragments/screens}.detail

import android.content.Context
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.mobil.modul3xml.data.ProblemRepository
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch
```

```
class DetailViewModel(
    private val repository: ProblemRepository =
ProblemRepository
) : ViewModel() {

    private val _state = MutableStateFlow(DetailState())
    val state: StateFlow<DetailState> = _state.asStateFlow()

    fun loadDetail(context: Context, problemId: String) {
        viewModelScope.launch(Dispatchers.IO) {
            val problem = repository.getProblemById(problemId)

            if (problem == null) {
                _state.update { it.copy(isNotFound = true) }
                return@launch
            }

            val codeText =
context.resources.openRawResource(problem.solutionCode).buffer
edReader().use { it.readText() }

            _state.update {
                it.copy(
                    titleRes = problem.title,
                    descRes = problem.description,
                    imgRes = problem.img,
                    code = codeText,
                )
            }
        }
    }
}
```

```

        }

    }

}

}

```

Tabel 5. HomeState Modul 3

```

package
com.mobil.modul3{xml/compose}.ui.{fragments/screens}.home

import com.mobil.modul3xml.data.CodeforcesProblem

data class HomeState(
    val problems: List<CodeforcesProblem> = emptyList()
)

```

Tabel 6. HomeViewModel Modul 3

```

package
com.mobil.modul3{xml/compose}.ui.{fragments/screens}.home

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.mobil.modul3xml.data.ProblemRepository
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

```

```

class HomeViewModel(
    private val repository: ProblemRepository =
ProblemRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow(HomeState())
    val uiState: StateFlow<HomeState> = _uiState.asStateFlow()

    init {
        loadProblems()
    }

    private fun loadProblems() {
        viewModelScope.launch(Dispatchers.IO) {
            val list = repository.getAllProblems()
            _uiState.update { it.copy(problems = list) }
        }
    }
}

```

Tabel 7. LanguageModel Modul 3

```

package com.mobil.modul3xml.ui.{fragments/screens}.language

data class LanguageModel(
    val name: String,
    val tag: String,

```

```
        val isSelected: Boolean
    )
}
```

Tabel 8. LanguageState Modul 3

```
package
com.mobil.modul3{xml/compose}.ui.{fragments/screens}.language

data class LanguageState(
    val selectedTag: String = "en",
    val languages: List<LanguageModel> = listOf(
        LanguageModel("English", "en", isSelected = true),
        LanguageModel("Indonesian", "id", isSelected = false)
    )
)
```

Tabel 9. LanguageViewModel Modul 3

```
package com.mobil.modul3xml.ui.fragments.language

import androidx.appcompat.app.AppCompatActivity
import androidx.core.os.LocaleListCompat
import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update

class LanguageViewModel : ViewModel() {
```

```

        private val _uiState = MutableStateFlow(LanguageState())

        val uiState: StateFlow<LanguageState> =
            _uiState.asStateFlow()

        init {

            loadCurrentLocale()

        }

        private fun loadCurrentLocale() {

            val locales =
                AppCompatDelegate.getApplicationLocales()

            val tag = if (!locales.isEmpty)
                locales[0]?.language ?: "en" else "en"

            _uiState.update { it.copy(selectedTag = tag) }

        }

        fun onLanguageSelected(tag: String) {

            val appLocale = LocaleListCompat.forLanguageTags(tag)

            AppCompatDelegate.setApplicationLocales(appLocale)

            _uiState.update { it.copy(selectedTag = tag) }

        }

    }

```

Compose

Tabel 10. DetailScreen Modul 3

```

package com.mobil.modul3compose.ui.screens.detail

import androidx.compose.foundation.Image

```

```
import androidx.compose.foundation.background
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import
androidx.compose.foundation.text.selection.SelectionContainer
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import
androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
```



```
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun DetailScreen(
    problemId: String,
    viewModel: DetailViewModel,
    onBack: () -> Unit
) {
    val context = LocalContext.current
    val state by viewModel.state.collectAsState()

    LaunchedEffect(problemId) {
        viewModel.loadDetail(context, problemId)
    }

    Scaffold(
        topBar = {
```

```

        TopAppBar(
            title = {
                val titleText = if (state.titleRes != 0)
stringResource(state.titleRes) else ""

                Text(titleText)
            },
            navigationIcon = {
                IconButton(onClick = onBackPressed) {
                    Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
                }
            }
        )
    }

) { paddingValues ->
    Box(modifier = Modifier.padding(paddingValues)) {
        if (state.isNotFound) {
            Box(Modifier.fillMaxSize(), contentAlignment =
Alignment.Center) {
                Text("Problem not found")
            }
        } else if (state.titleRes != 0) {
            Column(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(16.dp)
                    .verticalScroll(rememberScrollState())
            ) {

```

```

        Image(
            painterResource(state.imgRes),
            contentDescription = null,
            modifier =
Modifier.fillMaxWidth().height(128.dp).clip(RoundedCornerShape
(8.dp)),
            contentScale = ContentScale.Crop
        )
        Text(
            text = stringResource(state.titleRes),
            fontWeight = FontWeight.Bold,
            fontSize = 16.sp,
            modifier = Modifier.padding(top =
16.dp, bottom = 16.dp)
        )
        Text(
            text = stringResource(state.descRes),
            modifier = Modifier.padding(bottom =
16.dp)
        )
        CodePreview(state.code)
    }
}
}
}
}

@Composable

```

```

fun CodePreview(code: String) {

    val horizontalScrollState = rememberScrollState()

    SelectionContainer {

        Box(

            modifier = Modifier

                .fillMaxWidth()

                .clip(RoundedCornerShape(8.dp))

                .background(Color(0xFF1E1E1E))

                .padding(12.dp)

        ) {

            Text(

                text = code,

                color = Color(0xFFD4D4D4),

                fontFamily = FontFamily.Monospace,

                fontSize = 12.sp,

                softWrap = false,

                modifier =
Modifier.horizontalScroll(horizontalScrollState)

            )

        }

    }

}

```

Tabel 11. HomeScreen Modul 3

```

package com.mobil.modul3compose.ui.screens.home

```

```
import android.content.Intent
import androidx.compose.foundation.Image
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.layout.wrapContentHeight
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Language
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
```

```
import
androidx.compose.material3.carousel.HorizontalMultiBrowseCarousel

import
androidx.compose.material3.carousel.rememberCarouselState

import androidx.compose.runtime.Composable

import androidx.compose.runtime.collectAsState

import androidx.compose.runtime.getValue

import androidx.compose.ui.Alignment

import androidx.compose.ui.Modifier

import androidx.compose.ui.draw.clip

import androidx.compose.ui.layout.ContentScale

import androidx.compose.ui.platform.LocalContext

import androidx.compose.ui.res.painterResource

import androidx.compose.ui.res.stringResource

import androidx.compose.ui.text.font.FontWeight

import androidx.compose.ui.text.style.TextAlign

import androidx.compose.ui.unit.dp

import androidx.compose.ui.unit.sp

import androidx.navigation.NavController

import androidx.core.net.toUri

import androidx.lifecycle.viewmodel.compose.viewModel

import com.mobil.modul3compose.R

import com.mobil.modul3compose.data.CodeforcesProblem

import com.mobil.modul3compose.navigation.RoutingNames


@OptIn(ExperimentalMaterial3Api::class)

@Composable
```

```

fun HomeScreen(
    navController: NavController,
    viewModel: HomeViewModel = viewModel()
) {
    val state by viewModel.uiState.collectAsState()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(stringResource(id =
R.string.app_name)) },
                actions = {
                    IconButton(onClick =
{ navController.navigate(RoutingNames.LanguageScreen) }) {
                        Icon(
                            imageVector =
Icons.Default.Language,
                            contentDescription = "Change
Language"
                        )
                    }
                }
            )
        }
    ) { paddingValues ->
        HomeContent(
            problems = state.problems,
            onDetailClick = { id ->
navController.navigate(RoutingNames.DetailScreen(id)) },

```

```

        modifier = Modifier.padding(paddingValues)

    )

}

@Composable
private fun HomeContent(
    problems: List<CodeforcesProblem>,
    onDetailClick: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current

    LazyColumn(
        modifier = modifier.fillMaxSize(),
        contentPadding = PaddingValues(bottom = 16.dp)
    ) {
        if (problems.isNotEmpty()) {
            item {
                ProblemCarousel(
                    problems,
                    onDetailClick
                )
            }

            items(problems) { problem ->
                ProblemCard(
                    problem = problem,

```



```

        onExternalClick = {
            context.startActivity(Intent(Intent.ACTION_VIEW, problem.url.toUri()))
        },

        onDetailClick =
        { onDetailClick(problem.problemId) }

    )

}

}

}

```

@Composable

```

fun ProblemCard(
    problem: CodeforcesProblem,
    onExternalClick: () -> Unit,
    onDetailClick: () -> Unit
) {
    Card(
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant),
        modifier = Modifier.padding(8.dp)
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.padding(8.dp)
        ) {
            Image(

```

```

        painter = painterResource(id = problem.img),
        contentDescription = null,
        contentScale = ContentScale.Crop,
        modifier =
Modifier.width(128.dp).height(256.dp).clip(RoundedCornerShape(
16.dp))
    )
    Column {
        Text(
            text = stringResource(problem.title),
            modifier = Modifier.padding(16.dp),
            fontWeight = FontWeight.Bold
        )
        Text(
            text =
stringResource(problem.description),
            modifier = Modifier.padding(16.dp),
            textAlign = TextAlign.Justify,
            fontSize = 16.sp
        )
        Row(modifier = Modifier.fillMaxWidth(),
horizontalArrangement = Arrangement.End) {
            Button(onClick = onExternalClick, modifier
= Modifier.padding(8.dp)) {
                Text("Problem")
            }
            Button(onClick = onDetailClick, modifier =
Modifier.padding(8.dp)) {
                Text("Detail")
            }
        }
    }
}

```

```

    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProblemCarousel(
    problem: List<CodeforcesProblem>,
    onItemClick: (String) -> Unit
) {
    HorizontalMultiBrowseCarousel(
        state = rememberCarouselState { problem.count() },
        modifier = Modifier
            .fillMaxWidth()
            .wrapContentHeight()
            .padding(top = 16.dp, bottom = 16.dp),
        preferredItemWidth = 372.dp,
        itemSpacing = 8.dp,
        contentPadding = PaddingValues(horizontal = 16.dp)
    ) { i ->
        val item = problem[i]
        Image(
            modifier = Modifier
                .height(205.dp)
                .clickable { onItemClick(item.problemId) }
                .maskClip(MaterialTheme.shapes.extraLarge)

```

```

        .fillMaxSize(),

        painter = painterResource(item.img),

        contentDescription = stringResource(item.title),

        contentScale = ContentScale.Crop

    )

}

}

```

Tabel 12. LanguageScreen Modul 3

```

package com.mobil.modul3compose.ui.screens.language

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material.icons.Icons
import
androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.Check
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text

```

```
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.mobil.modul3compose.R

@Composable
fun LanguageScreen(
    navController: NavController,
    viewModel: LanguageViewModel = viewModel()
) {
    val state by viewModel.uiState.collectAsState()

    LanguageContent(
        state = state,
        onBackClick = { navController.popBackStack() },
        onLanguageClick = { viewModel.onLanguageSelected(it) }
    )
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
```

```

private fun LanguageContent(
    state: LanguageState,
    onBackClick: () -> Unit,
    onLanguageClick: (String) -> Unit
) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(stringResource(id =
R.string.language_title)) },
                navigationIcon = {
                    IconButton(onClick = onBackClick) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBa
ck, contentDescription = "Back")
                    }
                }
            )
        }
    ) { paddingValues ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
        ) {
            state.languages.forEach { language ->
                LanguageItem(
                    title = language.name,
                    tag = language.tag,

```

```

        currentTag = state.selectedTag,

        onClick = {
            { onLanguageClick(language.tag) }

        }

    }

}

@Composable
private fun LanguageItem(
    title: String,
    tag: String,
    currentTag: String,
    onClick: () -> Unit
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
            .padding(16.dp),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text = title)

        if (currentTag == tag) {
            Icon(Icons.Default.Check, contentDescription =
                "Selected")
        }
    }
}

```

```

    }

}

}

```

Tabel 13. AppNavigation Modul 3

```

package com.mobil.modul3compose.navigation

import androidx.compose.runtime.Composable
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavHostController
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.toRoute
import com.mobil.modul3compose.ui.screens.detail.DetailScreen
import
com.mobil.modul3compose.ui.screens.detail.DetailViewModel
import com.mobil.modul3compose.ui.screens.home.HomeScreen
import com.mobil.modul3compose.ui.screens.home.HomeViewModel
import
com.mobil.modul3compose.ui.screens.language.LanguageScreen
import
com.mobil.modul3compose.ui.screens.language.LanguageViewModel

@Composable
fun AppNavigation(navController: NavHostController) {
    NavHost(navController = navController, startDestination =
RoutingNames.HomeScreen) {
        composable<RoutingNames.HomeScreen>{

```



```

        val viewModel: HomeViewModel = viewModel()

        HomeScreen(navController, viewModel)
    }

    composable<RoutingNames.DetailScreen>{ navBackStack ->
        val detailArgs =
navBackStack.toRoute<RoutingNames.DetailScreen>()

        val viewModel: DetailViewModel = viewModel()

        DetailScreen(
            detailArgs.problemId,
            viewModel,
            { navController.popBackStack() }
        )
    }

    composable<RoutingNames.LanguageScreen> {
        val viewModel: LanguageViewModel = viewModel()

        LanguageScreen(navController, viewModel)
    }
}

```

Tabel 14. RoutingNames Modul 3

```

package com.mobil.modul3compose.navigation

import kotlinx.serialization.Serializable

@Serializable

```

```
sealed class RoutingNames {

    @Serializable
    object HomeScreen : RoutingNames()

    @Serializable
    data class DetailScreen(
        val problemId: String
    ) : RoutingNames()

    @Serializable
    object LanguageScreen: RoutingNames()
}
```

Tabel 15. MainActivity Compose Modul 3

```
package com.mobil.modul3compose

import android.os.Bundle
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.navigation.compose.rememberNavController
import com.mobil.modul3compose.navigation.AppNavigation
import com.mobil.modul3compose.ui.theme.Modul3ComposeTheme

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
```

```

        super.onCreate(savedInstanceState)

        enableEdgeToEdge()

        setContent {

            Modul3ComposeTheme {

                val navController = rememberNavController()

                AppNavigation(navController)

            }

        }

    }
}

```

XML

Tabel 16. DetailFragment Modul 3

```

package com.mobil.modul3xml.ui.fragments.detail

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.LifecycleScope
import androidx.lifecycle.RepeatOnLifecycle
import androidx.navigation.fragment.findNavController
import com.mobil.modul3xml.databinding.FragmentDetailBinding
import kotlinx.coroutines.launch

```

```
class DetailFragment : Fragment() {

    private var _binding: FragmentDetailBinding? = null
    private val binding get() = _binding!!
    private val viewModel: DetailViewModel by viewModels()

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentDetailBinding.inflate(inflater,
container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val problemId = arguments?.getString("problemId") ?:
return

        setupToolbar()

        observeViewModel()

        viewModel.loadDetail(requireContext(), problemId)
```

```

    }

    private fun setupToolbar() {
        binding.topAppBar.setNavigationOnClickListener {
            findNavController().navigateUp()
        }
    }

    private fun observeViewModel() {
        viewLifecycleOwner.lifecycleScope.launch {
            viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.state.collect { state ->
                    if (state.isNotFound) {
                        binding.tvNotFound.visibility =
View.VISIBLE
                        binding.contentScrollView.visibility =
View.GONE
                        binding.topAppBar.title = ""
                    } else if (state.titleRes != 0) {
                        binding.tvNotFound.visibility =
View.GONE
                        binding.contentScrollView.visibility =
View.VISIBLE
                        binding.topAppBar.title =
getString(state.titleRes)
                        binding.tvTitle.setText(state.titleRes
)
                    }
                }
            }
        }
    }

```

```

binding.tvDescription.setText(state.de
scRes)

binding.tvCode.text = state.code

if (state.imgRes != 0) {
    binding.ivHeader.setImageResource(
state.imgRes)
}
}
}
}
}
}

override fun onDestroyView() {
    super.onDestroyView()
    _binding = null
}
}

```

Tabel 17. ProblemListAdapter Modul 3

```

package com.mobil.modul3xml.ui.fragments.home

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.mobil.modul3xml.databinding.ItemProblemCardBinding

```

```

import com.mobil.modul3xml.data.CodeforcesProblem
import com.mobil.modul3xml.util.DiffCallback

class ProblemListAdapter(
    private val onExternalClick: (String) -> Unit,
    private val onDetailClick: (String) -> Unit
) : ListAdapter<CodeforcesProblem,
ProblemListAdapter.ProblemViewHolder>(
    DiffCallback<CodeforcesProblem> {it.problemId}
) {

    class ProblemViewHolder(
        private val binding: ItemProblemCardBinding,
        private val onExternalClick: (String) -> Unit,
        private val onDetailClick: (String) -> Unit
    ) : RecyclerView.ViewHolder(binding.root) {

        fun bind(problem: CodeforcesProblem) {
            binding.ivProblem.setImageResource(problem.img)
            binding.tvTitle.setText(problem.title)
            binding.tvDescription.setText(problem.description)

            binding.btnProblem.setOnClickListener
{ onExternalClick(problem.url) }

            binding.btnDetail.setOnClickListener
{ onDetailClick(problem.problemId) }

        }
    }
}

```

```

        override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): ProblemViewHolder {

            val binding = ItemProblemCardBinding.inflate(
                LayoutInflater.from(parent.context), parent, false
            )

            return ProblemViewHolder(binding, onExternalClick,
onDetailClick)

        }

        override fun onBindViewHolder(holder: ProblemViewHolder,
position: Int) {

            holder.bind(getItem(position))

        }

    }
}

```

Tabel 18. CarouselAdapter Modul 3

```

package com.mobil.modul3xml.ui.fragments.home

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.mobil.modul3xml.data.CodeforcesProblem
import
com.mobil.modul3xml.databinding.ItemProblemCarouselBinding
import com.mobil.modul3xml.util.DiffCallback

```



```

class CarouselAdapter(
    private val onItemClick: (String) -> Unit
) : ListAdapter<CodeforcesProblem,
CarouselAdapter.CarouselViewHolder>(
    DiffCallback<CodeforcesProblem> {it.problemId}
) {

    class CarouselViewHolder(
        private val binding: ItemProblemCarouselBinding,
        private val onItemClick: (String) -> Unit
    ) : RecyclerView.ViewHolder(binding.root) {

        fun bind(problem: CodeforcesProblem) {
            binding.ivCarouselItem.setImageResource(problem.im
g)
            binding.root.contentDescription =
binding.root.context.getString(problem.title)

            binding.root.setOnClickListener
{ onItemClick(problem.problemId) }
        }
    }

    override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): CarouselViewHolder {
        val binding = ItemProblemCarouselBinding.inflate(
            LayoutInflater.from(parent.context), parent, false
        )
        return CarouselViewHolder(binding, onItemClick)
    }
}

```

```

    }

    override fun onBindViewHolder(holder: CarouselViewHolder,
position: Int) {

        holder.bind(getItem(position))

    }
}

```

Tabel 19. HomeFragment Modul 3

```

package com.mobil.modul3xml.ui.fragments.home

import android.content.Intent
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.lifecyclescope
import androidx.lifecycle.repeatOnLifecycle
import androidx.navigation.fragment.findNavController
import com.mobil.modul3xml.R
import com.mobil.modul3xml.databinding.FragmentHomeBinding
import kotlinx.coroutines.launch
import androidx.core.net.toUri
import
com.google.android.material.carousel.CarouselLayoutManager

```

```
import com.google.android.material.carousel.CarouselSnapHelper
import
com.google.android.material.carousel.HeroCarouselStrategy

class HomeFragment : Fragment() {

    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!

    private val viewModel: HomeViewModel by viewModels()

    private lateinit var carouselAdapter: CarouselAdapter
    private lateinit var listAdapter: ProblemListAdapter

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater,
container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)
```

```

        setupToolbar()

        setupRecyclerViews()

        observeViewModel()
    }

    private fun setupToolbar() {
        binding.topAppBar.setOnMenuItemClickListener
    { menuItem ->
        when (menuItem.itemId) {
            R.id.action_language -> {
                findNavController().navigate(R.id.action_h
omeFragment_to_languageFragment)
                TRUE
            }
            else -> false
        }
    }
}

    private fun setupRecyclerViews() {
        val carouselLayoutManager =
CarouselLayoutManager(HeroCarouselStrategy())
        carouselLayoutManager.carouselAlignment =
CarouselLayoutManager.ALIGNMENT_START
        binding.carouselRecyclerView.layoutManager =
carouselLayoutManager

        val snapHelper = CarouselSnapHelper()
        snapHelper.attachToRecyclerView(binding.carouselRecycl

```

```

erView)

        carouselAdapter = CarouselAdapter { problemId ->
            navigateToDetail(problemId)
        }
        binding.carouselRecyclerView.adapter = carouselAdapter

        listAdapter = ProblemListAdapter(
            onExternalClick = { url ->
                val intent = Intent(Intent.ACTION_VIEW,
url.toUri())
                startActivity(intent)
            },
            onDetailClick = { problemId ->
                navigateToDetail(problemId)
            }
        )
        binding.listRecyclerView.adapter = listAdapter
    }

    private fun navigateToDetail(problemId: String) {
        val bundle = Bundle().apply{
            putString("problemId", problemId)
        }
        findNavController().navigate(R.id.action_homeFragment_
to_detailFragment, bundle)
    }

```

```

private fun observeViewModel() {
    viewLifecycleOwner.lifecycleScope.launch {
        viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
            viewModel.uiState.collect { state ->
                val problems = state.problems
                carouselAdapter.submitList(problems)
                listAdapter.submitList(problems)

                if (problems.isEmpty()) {
                    binding.carouselRecyclerView.visibility = View.GONE
                    binding.listRecyclerView.visibility = View.GONE
                } else {
                    binding.carouselRecyclerView.visibility = View.VISIBLE
                    binding.listRecyclerView.visibility = View.VISIBLE
                }
            }
        }
    }
}

override fun onDestroyView() {
    super.onDestroyView()
    _binding = null
}

```

```
}
```

Tabel 20. LanguageAdapter Modul 3

```
package com.mobil.modul3xml.ui.fragments.language

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.mobil.modul3xml.databinding.ItemLanguageBinding
import com.mobil.modul3xml.util.DiffCallback

class LanguageAdapter(
    private val onLanguageClick: (String) -> Unit
) : ListAdapter<LanguageModel,
LanguageAdapter.LanguageViewHolder>(
    DiffCallback<LanguageModel> { it.tag }
) {
    class LanguageViewHolder(private val binding:
ItemLanguageBinding) :
        RecyclerView.ViewHolder(binding.root) {

        fun bind(item: LanguageModel, onLanguageClick:
(String) -> Unit) {
            binding.tvLanguageName.text = item.name
            binding.ivCheck.visibility = if (item.isSelected)
View.VISIBLE else View.INVISIBLE
        }
    }
}
```

```

        binding.root.setOnClickListener {
            onLanguageClick(item.tag)
        }
    }
}

override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): LanguageViewHolder {
    val binding = ItemLanguageBinding.inflate(
        LayoutInflater.from(parent.context), parent, false
    )
    return LanguageViewHolder(binding)
}

override fun onBindViewHolder(holder: LanguageViewHolder,
position: Int) {
    holder.bind(getItem(position), onLanguageClick)
}
}

```

Tabel 21. LanguageFragment Modul 3

```

package com.mobil.modul3xml.ui.fragments.language

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup

```



```
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.lifecycleScope
import androidx.lifecycle.repeatOnLifecycle
import androidx.navigation.fragment.findNavController
import com.mobil.modul3xml.databinding.FragmentLanguageBinding
import kotlinx.coroutines.launch

class LanguageFragment : Fragment() {

    private var _binding: FragmentLanguageBinding? = null
    private val binding get() = _binding!!

    private val viewModel: LanguageViewModel by viewModels()
    private lateinit var languageAdapter: LanguageAdapter

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentLanguageBinding.inflate(inflater,
container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
```

```

Bundle?) {

    super.onViewCreated(view, savedInstanceState)

    setupToolbar()

    setupRecyclerView()

    observeViewModel()

}

private fun setupToolbar() {

    binding.topAppBar.setNavigationOnClickListener {

        findNavController().navigateUp()

    }

}

private fun setupRecyclerView() {

    languageAdapter = LanguageAdapter { tag ->

        viewModel.onLanguageSelected(tag)

    }

    binding.rvLanguages.adapter = languageAdapter

}

private fun observeViewModel() {

    viewLifecycleOwner.lifecycleScope.launch {

        viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {

            viewModel.uiState.collect { state ->

                val uiModels = state.languages.map
{ language ->

```

```

        LanguageModel(
            name = language.name,
            tag = language.tag,
            isSelected = language.tag ==
state.selectedTag
        )
    }
    languageAdapter.submitList(uiModels)
}
}
}
}

override fun onDestroyView() {
    super.onDestroyView()
    _binding = null
}
}

```

Tabel 22. DiffCallback Modul 3

```

package com.mobil.modul3xml.util

import androidx.recyclerview.widget.DiffUtil

class DiffCallback<T : Any>(
    private val itemIdentifier: (T) -> Any
) : DiffUtil.ItemCallback<T>() {

```

```

        override fun areItemsTheSame(oldItem: T, newItem: T):
Boolean {

                                return    itemIdentifier(oldItem)    ==
itemIdentifier(newItem)

        }

        override fun areContentsTheSame(oldItem: T, newItem: T):
Boolean {

            return oldItem == newItem

        }
    }
}

```

Tabel 23. MainActivity XML Modul 3

```

package com.mobil.modul3xml

import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import com.mobil.modul3xml.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {

```

```

        super.onCreate(savedInstanceState)

        enableEdgeToEdge()

        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        ViewCompat.setOnApplyWindowInsetsListener(binding.root
    ) { v, insets ->

                                val systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars())

                                v.setPadding(systemBars.left, systemBars.top,
systemBars.right, systemBars.bottom)

                                insets

        }

    }
}

```

XML Layout

Tabel 24. MainActivity XML Modul 3

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <androidx.fragment.app.FragmentContainerView

```

```

        android:id="@+id/nav_host_fragment"

        android:name="androidx.navigation.fragment.NavHostFragment"

        android:layout_width="match_parent"

        android:layout_height="match_parent"

        app:defaultNavHost="true"

        app:navGraph="@navigation/nav_graph" />

</LinearLayout>

```

Tabel 25. FragmentDetail XML Modul 3

```

<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:app="http://schemas.android.com/apk/res-auto"

    xmlns:tools="http://schemas.android.com/tools"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    tools:context=".ui.fragments.detail.DetailFragment">

    <com.google.android.material.appbar.AppBarLayout

        android:id="@+id/appBarLayout"

        android:layout_width="0dp"

        android:layout_height="wrap_content"

        app:layout_constraintEnd_toEndOf="parent"

        app:layout_constraintStart_toStartOf="parent"

        app:layout_constraintTop_toTopOf="parent">

```

```
<com.google.android.material.appbar.MaterialToolbar
    android:id="@+id/topAppBar"
    android:layout_width="match_parent"
    android:layout_height="?attr/actionBarSize"
    app:navigationIcon="@drawable/ic_arrow_back"
    tools:title="Problem Title" />
</com.google.android.material.appbar.AppBarLayout>
```

```
<TextView
    android:id="@+id/tvNotFound"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/problem_not_found"
    android:visibility="gone"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/appBarLayout"
    tools:visibility="visible" />
```

```
<androidx.core.widget.NestedScrollView
    android:id="@+id/contentScrollView"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:padding="16dp"
    android:visibility="gone"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
```

```

        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/appBarLayout"
        tools:visibility="visible">

        <LinearLayout

            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <com.google.android.material.imageview.ShapeableIm
ageView

                android:id="@+id/ivHeader"
                android:layout_width="match_parent"
                android:layout_height="128dp"
                android:scaleType="centerCrop"
                app:shapeAppearanceOverlay="@style/ShapeAppear
anceOverlay.App.CornerRadius8dp"
                tools:src="@tools:sample/backgrounds/scenic" /
            >

            <TextView

                android:id="@+id/tvTitle"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="16dp"
                android:layout_marginBottom="16dp"
                android:textSize="16sp"
                android:textStyle="bold"

```



```
        tools:text="Problem Title" />

<TextView

    android:id="@+id/tvDescription"

    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:layout_marginBottom="16dp"

        tools:text="Problem description goes
here..." />

<com.google.android.material.card.MaterialCardView

    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    app:cardBackgroundColor="#1E1E1E"

    app:cardCornerRadius="8dp"

    app:strokeWidth="0dp">

    <HorizontalScrollView

        android:layout_width="match_parent"

        android:layout_height="wrap_content"

        android:fillViewport="true"

        android:scrollbars="none">

        <TextView

            android:id="@+id/tvCode"

            android:layout_width="wrap_content"

            android:layout_height="wrap_content"

            android:fontFamily="monospace"
```

```

        android:padding="12dp"

        android:textColor="#D4D4D4"

        android:textIsSelectable="true"

        android:textSize="12sp"

        tools:text="fun main() {\n
println() \n}" />

    </HorizontalScrollView>

</com.google.android.material.card.MaterialCardView>

</LinearLayout>

</androidx.core.widget.NestedScrollView>

</androidx.constraintlayout.widget.ConstraintLayout>

```

Tabel 26. FragmentHome Modul 3

```

<?xml version="1.0" encoding="utf-8"?>

<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ui.fragments.home.HomeFragment">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

```

```
<com.google.android.material.appbar.MaterialToolbar
    android:id="@+id/topAppBar"
    android:layout_width="match_parent"
    android:layout_height="?attr/actionBarSize"
    app:title="@string/app_name"
    app:menu="@menu/home_menu" />
</com.google.android.material.appbar.AppBarLayout>

<androidx.core.widget.NestedScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layout_behavior="@string/appbar_scrolling_view_beh
avior">

    <LinearLayout

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:paddingBottom="16dp">

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/carouselRecyclerView"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:layout_marginBottom="16dp"
            android:clipToPadding="false"
```

```

        android:paddingHorizontal="16dp"

        tools:listitem="@layout/item_problem_carousel"
/>

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/listRecyclerView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:nestedScrollingEnabled="false"
    app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"
    tools:listitem="@layout/item_problem_card" />

</LinearLayout>

</androidx.core.widget.NestedScrollView>

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

Tabel 27. FragmentLanguage XML Modul 3

```

<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".ui.fragments.language.LanguageFragment">

```

```

<com.google.android.material.appbar.AppBarLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/topAppBar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        app:navigationIcon="@drawable/ic_arrow_back"
        app:title="@string/language_title" />
</com.google.android.material.appbar.AppBarLayout>

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rvLanguages"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layoutManager="androidx.recyclerview.widget.Linear
LayoutManager"
    tools:listitem="@layout/item_language" />

</LinearLayout>

```

Tabel 28. ItemLanguage XML Modul 3

```

<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"

```

```
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="?attr/selectableItemBackground"
        android:clickable="true"
        android:focusable="true"
        android:gravity="center_vertical"
        android:orientation="horizontal"
        android:padding="16dp">

        <TextView
            android:id="@+id/tvLanguageName"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:textAppearance="?attr/textAppearanceBodyLarge"
            tools:text="English" />

        <ImageView
            android:id="@+id/ivCheck"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:src="@drawable/ic_check"
            android:visibility="invisible"
            tools:visibility="visible" />
    </LinearLayout>
```

Tabel 29. ItemProblemCard XML Modul 3

```
<?xml version="1.0" encoding="utf-8"?>

<com.google.android.material.card.MaterialCardView

    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="8dp"
    android:layout_marginVertical="8dp"
    app:cardBackgroundColor="?attr/colorSurfaceVariant"
    app:cardCornerRadius="16dp">

    <androidx.constraintlayout.widget.ConstraintLayout

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="8dp">

        <com.google.android.material.imageview.ShapeableImageV
        iew

            android:id="@+id/ivProblem"
            android:layout_width="128dp"
            android:layout_height="256dp"
            android:scaleType="centerCrop"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            app:shapeAppearanceOverlay="@style/ShapeAppearance
```

```
Overlay.App.CornerRadius16Percent"
```

```
tools:src="@tools:sample/backgrounds/scenic" />
```

```
<TextView
```

```
android:id="@+id/tvTitle"
```

```
android:layout_width="0dp"
```

```
android:layout_height="wrap_content"
```

```
android:layout_marginStart="16dp"
```

```
android:layout_marginTop="8dp"
```

```
android:layout_marginEnd="16dp"
```

```
android:textStyle="bold"
```

```
app:layout_constraintEnd_toEndOf="parent"
```

```
app:layout_constraintStart_toEndOf="@id/ivProblem"
```

```
app:layout_constraintTop_toTopOf="parent"
```

```
tools:text="Problem Title" />
```

```
<TextView
```

```
android:id="@+id/tvDescription"
```

```
android:layout_width="0dp"
```

```
android:layout_height="wrap_content"
```

```
android:layout_marginStart="16dp"
```

```
android:layout_marginTop="16dp"
```

```
android:layout_marginEnd="16dp"
```

```
android:justificationMode="inter_word"
```

```
android:textSize="16sp"
```

```
app:layout_constraintEnd_toEndOf="parent"
```

```
app:layout_constraintStart_toEndOf="@id/ivProblem"
```

```
app:layout_constraintTop_toBottomOf="@id/tvTitle"
```



```

        tools:text="Problem description goes here" />

<Button
    android:id="@+id/btnProblem"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginEnd="8dp"
    android:text="@string/problem"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toStartOf="@id/btnDetail"
    app:layout_constraintTop_toBottomOf="@id/tvDescription"
    app:layout_constraintVertical_bias="1.0" />

<Button
    android:id="@+id/btnDetail"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/detail"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvDescription"
    app:layout_constraintVertical_bias="1.0" />

</androidx.constraintlayout.widget.ConstraintLayout>
</com.google.android.material.card.MaterialCardView>

```

Tabel 30. ItemProblemCarousel XML Modul 3

```
<?xml version="1.0" encoding="utf-8"?>

<com.google.android.material.carousel.MaskableFrameLayout

    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="205dp"
    android:layout_marginEnd="8dp"

    app:shapeAppearanceOverlay="@style/ShapeAppearanceOverlay.
App.CornerRadiusExtraLarge">

    <ImageView

        android:id="@+id/ivCarouselItem"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scaleType="centerCrop"
        tools:src="@tools:sample/backgrounds/scenic" />

</com.google.android.material.carousel.MaskableFrameLayout>
```

Tabel 31. HomeMenu XML Modul 3

```
<?xml version="1.0" encoding="utf-8"?>

<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item

        android:id="@+id/action_language"
```

```

        android:icon="@drawable/ic_language"

        android:title="Language"

        app:showAsAction="ifRoom" />
    </menu>

```

XML Nav Graph

Tabel 32. NavGraph XML Modul 3

```

<?xml version="1.0" encoding="utf-8"?>
<navigation
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/homeFragment">

    <fragment
        android:id="@+id/homeFragment"
        android:name="com.mobil.modul3xml.ui.fragments.home.HomeFragment"
        android:label="@string/app_name"
        tools:layout="@layout/fragment_home">

        <action
            android:id="@+id/action_homeFragment_to_detailFragment"
            app:destination="@id/detailFragment" />

        <action

```

```

        android:id="@+id/action_homeFragment_to_languageFr
        agment"

        app:destination="@id/languageFragment" />

    </fragment>

    <fragment

        android:id="@+id/detailFragment"

        android:name="com.mobil.modul3xml.ui.fragments.detail.
        DetailFragment"

        android:label="Detail"

        tools:layout="@layout/fragment_detail">

        <argument

            android:name="problemId"

            app:argType="string" />

        </fragment>

    <fragment

        android:id="@+id/languageFragment"

        android:name="com.mobil.modul3xml.ui.fragments.languag
        e.LanguageFragment"

        android:label="Language"

        tools:layout="@layout/fragment_language">

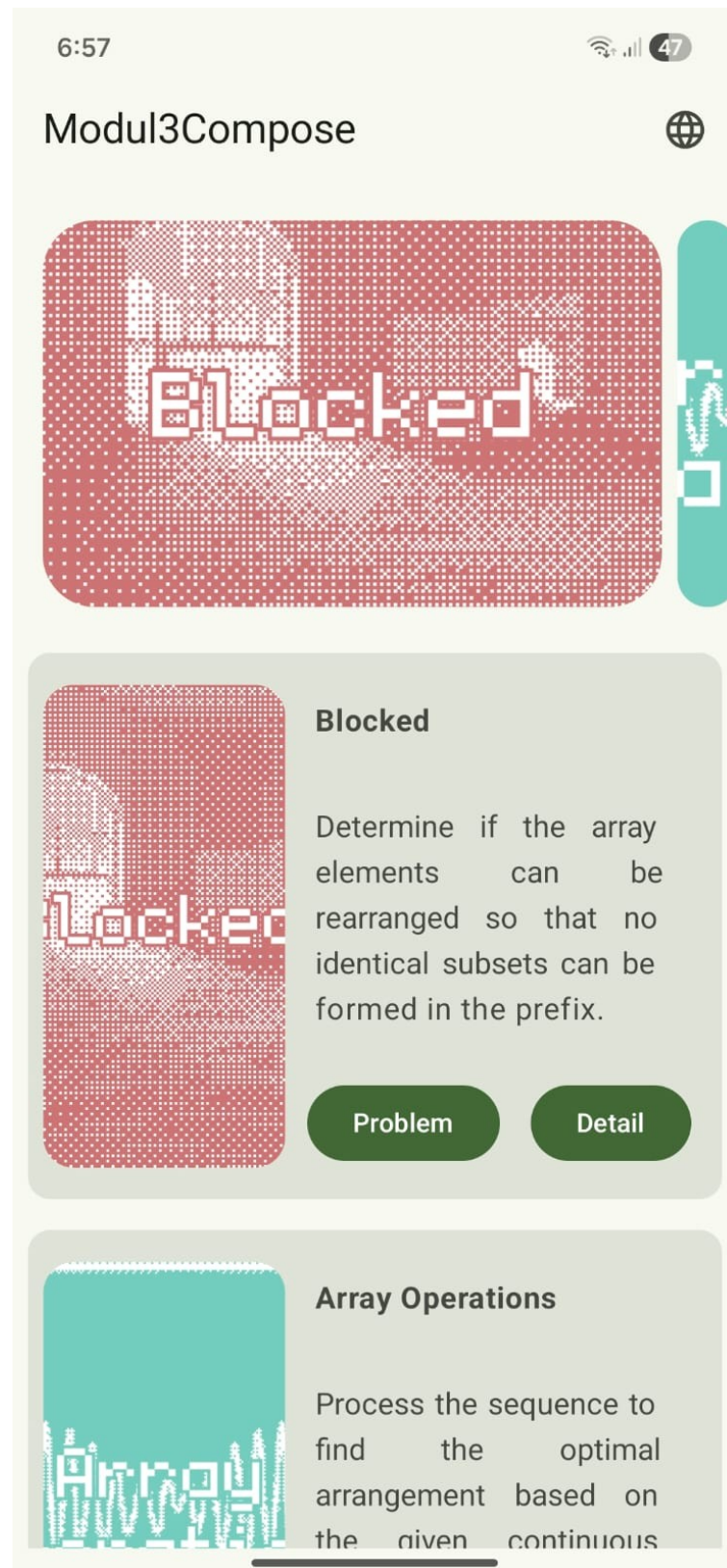
    </fragment>

</navigation>

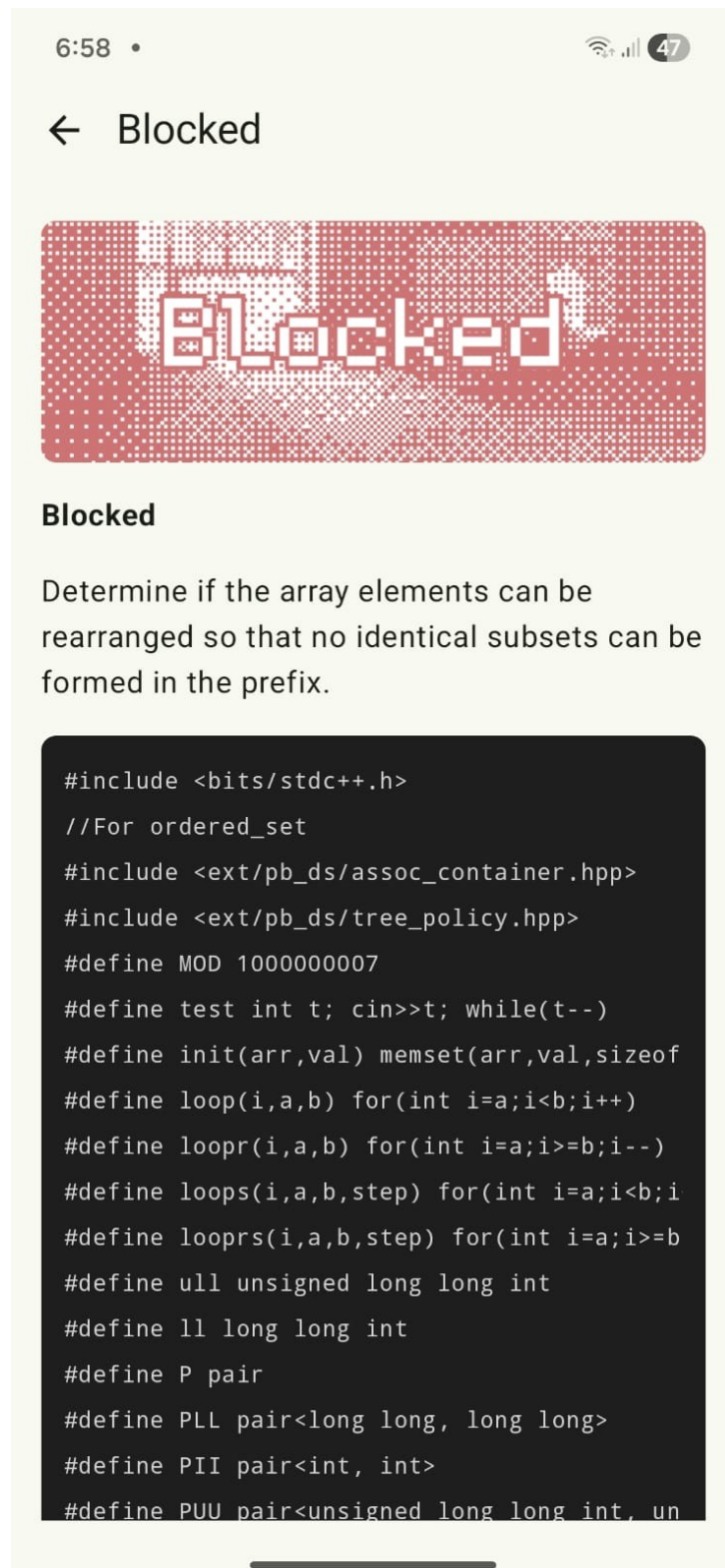
```

B. Output Program

Compose



Gambar 1. Home Page Compose English Modul 3



Gambar 2. Detail Page Compose English Modul 3

6:58 •



← Language

English



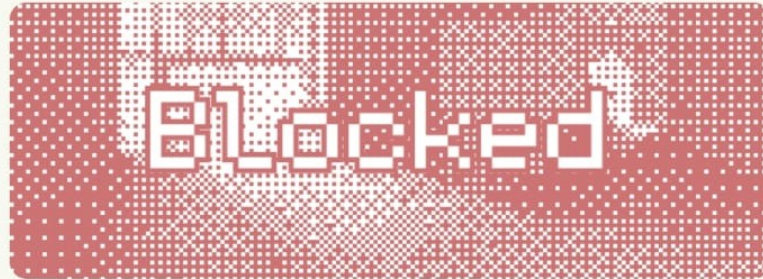
Indonesian

Gambar 3. Language Page Compose English Modul 3



Gambar 4. Home Page Compose Indonesia Modul 3

← Blocked

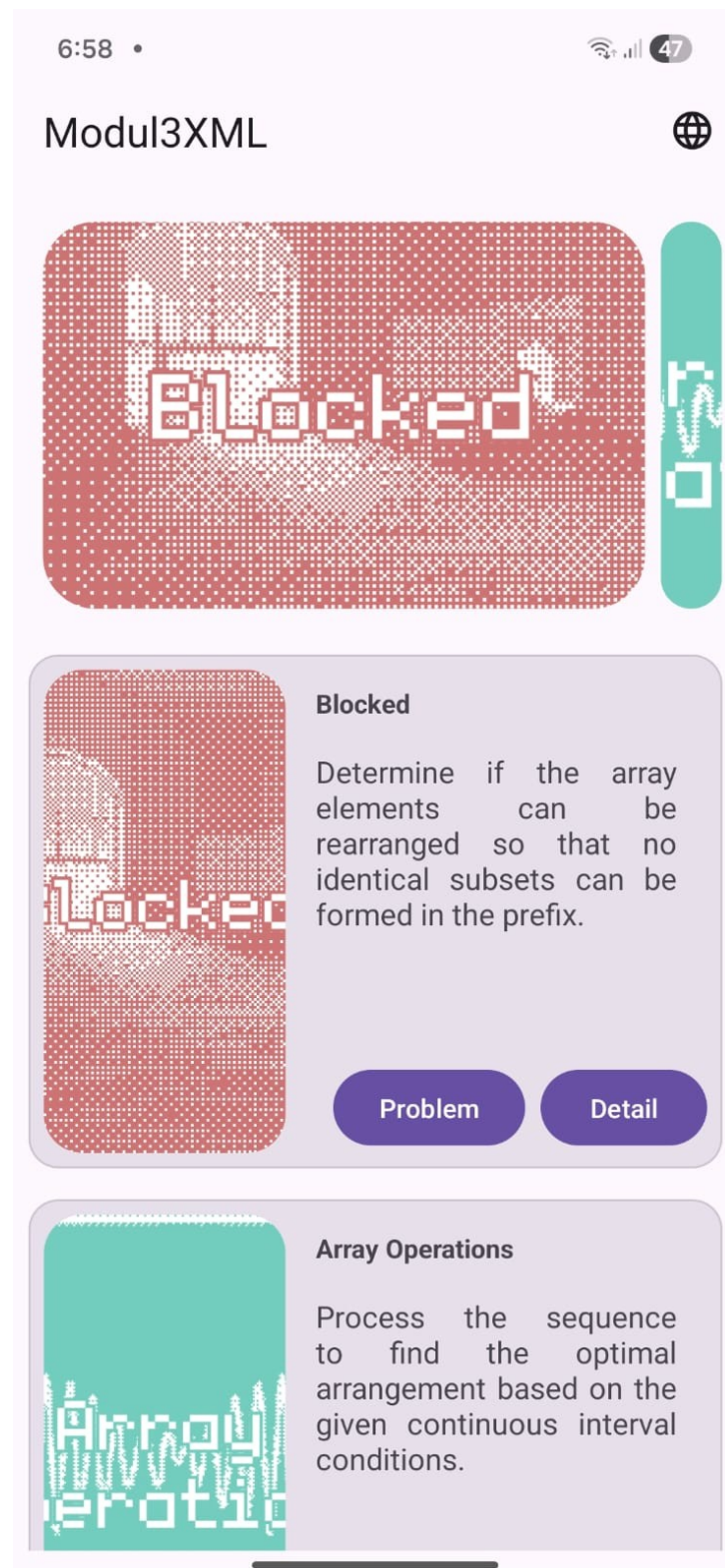


Blocked

Tentukan apakah elemen array dapat disusun ulang sehingga tidak ada subhimpunan identik yang terbentuk pada prefiks.

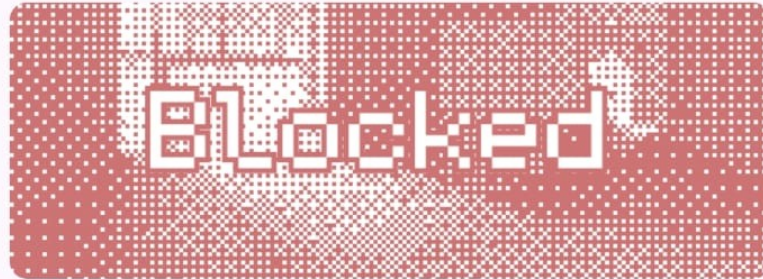
```
#include <bits/stdc++.h>
//For ordered_set
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#define MOD 1000000007
#define test int t; cin>>t; while(t--)
#define init(arr,val) memset(arr,val,sizeof
#define loop(i,a,b) for(int i=a;i<b;i++)
#define loopr(i,a,b) for(int i=a;i>=b;i--)
#define loops(i,a,b,step) for(int i=a;i<b;i
#define looprs(i,a,b,step) for(int i=a;i>=b
#define ull unsigned long long int
#define ll long long int
#define P pair
#define PLL pair<long long, long long>
#define PII pair<int, int>
#define PUU pair<unsigned long long int, un
```

Gambar 5. Detail Page Compose Indonesia Modul 3



Gambar 6. Home Page XML English Modul 3

← Blocked



Blocked

Determine if the array elements can be rearranged so that no identical subsets can be formed in the prefix.

```
#include <bits/stdc++.h>
//For ordered_set
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#define MOD 1000000007
#define test int t; cin>>t; while(t--)
#define init(arr,val) memset(arr,val,sizeof(arr))
#define loop(i,a,b) for(int i=a;i<b;i++)
#define loopr(i,a,b) for(int i=a;i>=b;i--)
#define loops(i,a,b,step) for(int i=a;i<b;i+=step)
#define looprs(i,a,b,step) for(int i=a;i>=b;i-=step)
#define ull unsigned long long int
#define ll long long int
#define P pair
#define PLL pair<long long, long long>
#define PII pair<int, int>
#define PUU pair<unsigned long long int, unsigned long long int>
#define L list
#define V vector
#define D deque
#define ST set
#define MS multiset
#define M map
#define UM unordered_map
#define mp make_pair
#define pb push_back
#define pf push_front
#define MM multimap
#define F first
#define S second
#define IT iterator
```

Gambar 7. Detail Page XML English Modul 3

6:58 •



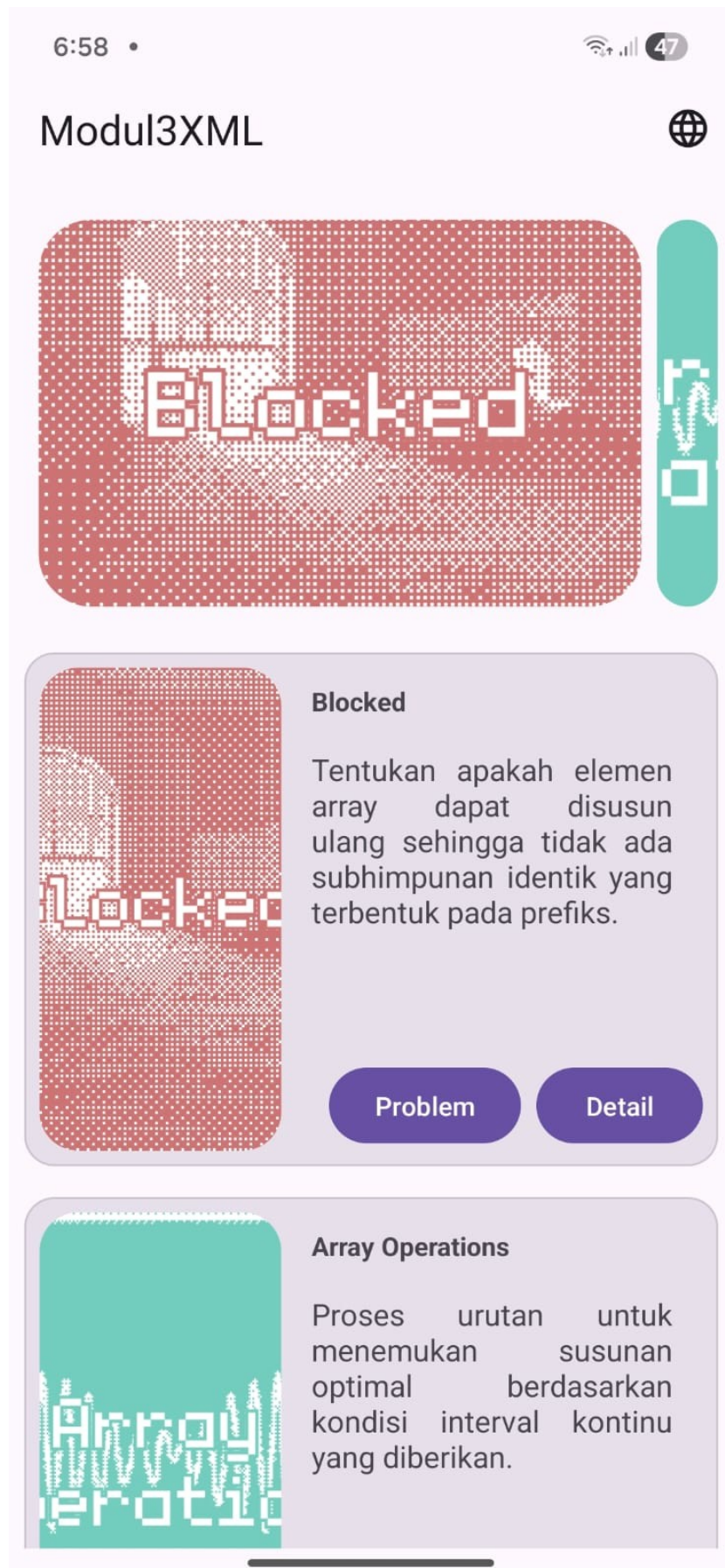
← Language

English



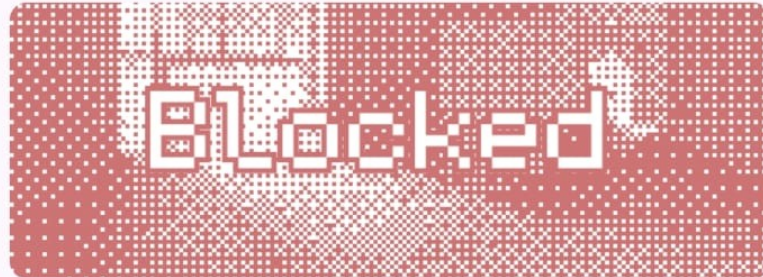
Indonesian

Gambar 8. Language Page XML English Modul 3



Gambar 9. Home Page XML Indonesia Modul 3

← Blocked



Blocked

Tentukan apakah elemen array dapat disusun ulang sehingga tidak ada subhimpunan identik yang terbentuk pada prefiks.

```
#include <bits/stdc++.h>
//For ordered_set
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#define MOD 1000000007
#define test int t; cin>>t; while(t--)
#define init(arr,val) memset(arr,val,sizeof(arr))
#define loop(i,a,b) for(int i=a;i<b;i++)
#define loopr(i,a,b) for(int i=a;i>=b;i--)
#define loops(i,a,b,step) for(int i=a;i<b;i+=step)
#define looprs(i,a,b,step) for(int i=a;i>=b;i-=step)
#define ull unsigned long long int
#define ll long long int
#define P pair
#define PLL pair<long long, long long>
#define PII pair<int, int>
#define PUU pair<unsigned long long int, unsigned long long int>
#define L list
#define V vector
#define D deque
#define ST set
#define MS multiset
#define M map
#define UM unordered_map
#define mp make_pair
#define pb push_back
#define pf push_front
#define MM multimap
#define F first
#define S second
```

Gambar 10. Detail Page XML Indonesia Modul 3

C. Pembahasan

Generic

Data

Saya menggunakan resource untuk menyimpan hampir semua data dari title, description dan image. Untuk solutionCode saya menggunakan raw (c++) supaya saya hanya perlu mengimport filenya saja tanpa diedit di dalam string.xml atau source code kotlin. Alasan kenapa saya menggunakan string resource untuk hampir semua data text adalah untuk mendukung fitur multi bahasa.

ViewModel

ViewModel (Home, Detail, Language) di Compose dan XML merupakan implementasi yang sama karena saya mendecouple-nya dari UI. Jadi, ketika saya migrasi dari Compose ke XML saya hanya perlu mengubah implementasi UI-nya saja.

Compose

MainActivity

MainActivity mewarisi AppCompatActivity bukan ComponentActivity karena dibutuhkan untuk mendukung AppCompatActivity yang digunakan pada fitur multi bahasa. enableEdgeToEdge() dipanggil agar tampilan melebar hingga ke system bar. NavController dibuat di sini menggunakan rememberNavController() lalu diteruskan ke AppNavigation, supaya lifecycle navigasi tetap terikat pada Activity. Hal ini membuat MainActivity menjadi entry point yang langsing no god class karena seluruh logika navigasi dan UI dipisahkan sepenuhnya ke Compose.

Home Screen

HomeScreen dipisah menjadi dua fungsi composable, yaitu HomeScreen yang menangani navigasi dan HomeContent yang menampilkan UI, saya pisah supaya responsibility antar komponen menjadi lebih pure. Daftar problem ditampilkan menggunakan LazyColumn untuk efisiensi rendering, dan dilengkapi HorizontalMultiBrowseCarousel di bagian atas sebagai navigasi visual dan karena diminta oleh laprak. Aksi membuka URL eksternal Codeforces

ditangani langsung dengan `Intent.ACTION_VIEW` di dalam `HomeContent` menggunakan `LocalContext`, tanpa perlu meneruskan context ke `ViewModel`. Hal ini menjaga `ViewModel` tetap bebas dari dependency pada Android framework context.

Detail Screen

`DetailScreen` menggunakan `LaunchedEffect(problemId)` supaya update data hanya ketika `problemId` berubah, jadi tidak ada pemanggilan ulang yang tidak perlu saat rekomposisi. `SolutionCode` ditampilkan melalui composable `CodePreview` yang terpisah, dengan support horizontal scroll dan `SelectionContainer` agar pengguna bisa menyalin kode dengan mudah. Kondisi `isNotFound` dan `titleRes == 0` digunakan sebagai indikator loading dan error state secara tidak eksplisit atau implisit tanpa memerlukan state tambahan. Pemisahan `onBack` sebagai lambda membuat `DetailScreen` tidak dependen pada `NavController`, sehingga lebih mudah diuji dan di-reuse.

Language Screen

`LanguageScreen` memisahkan antara lapisan navigasi dan lapisan UI dengan membagi composable menjadi `LanguageScreen` yang memegang `NavController` dan `LanguageContent` yang hanya menerima state dan callback. Setiap pilihan bahasa dirender menggunakan `forEach` karena jumlah bahasanya statis dan kecil, sehingga `LazyColumn` tidak diperlukan. Indikator bahasa yang sedang aktif ditampilkan dengan ikon centang melalui perbandingan `currentTag == tag`, memberikan feedback visual yang langsung dan jelas kepada pengguna. Pergantian bahasa ditangani sepenuhnya oleh `LanguageViewModel` menggunakan `AppCompatDelegate`, sehingga UI tidak perlu mengetahui mekanisme internal perubahan locale.

Navigation

`AppNavigation` menggunakan type-safe navigation dari Jetpack Navigation Compose dengan sealed class `RoutingNames` yang di-annotate dengan `@Serializable`, menghindari penggunaan string langsung yang rawan typo dan menghindari 2 jam debugging. Setiap composable di dalam `NavHost` langsung membuat instance `ViewModel`-nya masing-masing menggunakan `viewModel()`, sehingga lifecycle `ViewModel` selaras dengan back stack entry yang sesuai. Argumen navigasi untuk `DetailScreen` diekstrak menggunakan `toRoute<>()` yang

memanfaatkan type-safety penuh tanpa perlu parsing manual dari Bundle. Pendekatan ini membuat graf navigasi menjadi satu-satunya tempat yang mengetahui hubungan antar screen, sementara masing-masing screen tetap independen satu sama lain.

XML

MainActivity

Pada XML `enableEdgeToEdge()` tetap digunakan dan padding system bar ditangani secara manual melalui `ViewCompat.setOnApplyWindowInsetsListener`, sehingga konten tidak tertutup oleh status bar maupun navigation bar. Penggunaan `ViewBinding` memastikan akses ke view dilakukan secara type-safe tanpa `findViewById`. Dengan pola ini, `MainActivity` pure bertugas sebagai container navigasi tanpa menyimpan logika bisnis apapun.

Home Fragment

`HomeFragment` menggunakan dua `RecyclerView` terpisah, yaitu satu untuk carousel dengan `CarouselLayoutManager + HeroCarouselStrategy` dan satu untuk list problem dengan `LinearLayoutManager`, keduanya di-submit data yang sama dari `ViewModel` secara bersamaan. Kedua adapter yaitu `CarouselAdapter` dan `ProblemListAdapter` mewarisi `ListAdapter` yang memanfaatkan `DiffCallback`, sehingga update list dilakukan secara efisien dengan animasi diff otomatis tanpa perlu `notifyDataSetChanged()`. Observer state menggunakan `repeatOnLifecycle(STARTED)` untuk memastikan collection Flow hanya aktif saat Fragment benar-benar terlihat, menghindari pemborosan resource yang banyak di background. Navigasi ke `DetailFragment` diimplementasikan dalam satu fungsi `navigateToDetail()` agar mengikuti konsep DRY untuk navigasi dari problem carousel dan dari problem card.

Detail Fragment

`DetailFragment` mengambil `problemId` dari arguments yang dikirim melalui Bundle saat navigasi, lalu langsung memanggil `viewModel.loadDetail()` di `onViewCreated` sehingga data mulai di-fetch begitu view siap. Tampilan konten dan pesan “not found” dikontrol dengan mengubah visibility secara eksplisit atau tidak implisit berdasarkan state dari `ViewModel`, pola yang setara dengan kondisi `isNotFound` dan `titleRes == 0` di versi Compose. `SolutionCode` ditampilkan di dalam `HorizontalScrollView` yang memiliki anak (child)

TextView dengan `textIsSelectable="true"`, memberikan anaknya ability untuk scroll horizontal dan text selection yang fungsionalitasnya sama persis dengan CodePreview di versi Compose. Binding di-null-kan di `onDestroyView()` untuk mencegah memory leak yang umum terjadi pada Fragment dengan ViewBinding.

Language Fragment

LanguageFragment menggunakan RecyclerView dengan LanguageAdapter alih-alih `forEach` seperti di versi Compose, karena RecyclerView lebih sesuai dengan paradigma XML dan tetap efisien untuk daftar yang mungkin bertambah di masa depan. Status bahasa yang dipilih direpresentasikan melalui field `isSelected` di LanguageModel yang dihasilkan dari mapping di observer, sehingga LanguageAdapter tidak perlu mengetahui `selectedTag` secara langsung dan tugasnya pure untuk menampilkan data. Perubahan bahasa tetap ditangani oleh LanguageViewModel menggunakan `AppCompatDelegate`, konsisten dengan versi Compose, sehingga Fragment tidak terlibat dalam logika pergantian locale sama sekali. Pola setup yang dipisahkan menjadi `setupToolbar()`, `setupRecyclerView()`, dan `observeViewModel()` membuat `onViewCreated` tetap clean dan mudah dibaca oleh anak berusia 5 tahun.

Nav Graph

`nav_graph.xml` mendefinisikan seluruh struktur navigasi aplikasi dalam satu file XML, setara dengan peran `AppNavigation.kt` di versi Compose. Argumen `problemId` untuk DetailFragment dideklarasikan langsung di dalam tag `<fragment>` dengan `app:argType="string"`, sehingga janji manis atau contract antar destination terdokumentasi di satu tempat yang terpusat. Navigasi antar fragment dilakukan menggunakan action ID seperti `R.id.action_homeFragment_to_detailFragment`, yang meskipun tidak `se-type-safe` `toRoute<>()` di Compose, tetap terstruktur dan tidak tersebar di berbagai tempat karena semua navigasi dikelola dari fragment asal masing-masing. Pendekatan ini memungkinkan Android Studio untuk memvisualisasikan graf navigasi secara visual, yang memudahkan pemahaman alur aplikasi bahkan untuk orang bisnis sekalipun.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahasan

RecyclerView masih banyak digunakan karena di XML memang tidak ada alternatif lain yang setara dengan LazyColumn, berbeda dengan Compose yang menyediakan LazyColumn dengan paradigma deklaratif dan tanpa boilerplate seperti Adapter maupun DiffCallback. Jika konteksnya adalah pengembangan berbasis Compose, LazyColumn jelas menjadi pilihan utama saya karena kodenya lebih ringkas dan lebih menyenangkan untuk dikoding, ketika saya ngoding pakai Compose saya mendapatkan dopamin yang tinggi dan low cortisol.

Namun XML sendiri masih banyak digunakan di industri karena migrasi ke Compose tidak selalu worth it, terutama pada proyek yang sudah berjalan lama dan terlalu besar. Biaya migrasi dari sisi waktu dan resiko performa melambat sering kali dianggap lebih besar daripada manfaat yang didapat, sehingga kata orang bisnis keputusan untuk tetap bertahan di XML masih sangat masuk akal.

SOAL 3

Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut pendapat kalian, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

A. Pembahasan

Menurut saya Clean Architecture harus diterapkan karena memisahkan logika bisnis dari UI, sehingga setiap modul memiliki responsibility yang jelas dan perubahan di satu sisi tidak merembet ke sisi lain. Bukti nyatanya dapat terlihat di proyek ini, di mana ViewModel yang sama berhasil digunakan di dua versi UI dengan paradigma yang berbeda yaitu Compose dan XML, tanpa perubahan apapun pada logika bisnisnya. Selain manfaat teknisnya, kode yang terstruktur rapi juga mengurangi beban kognitif developer sehingga menghasilkan developer yang low cortisol dan high dopamin, alhasil meningkatkan produktivitas dan pada akhirnya memberikan dampak positif terhadap kualitas produk secara keseluruhan.

LINK GITHUB

<https://github.com/Gunturadhtya/Praktikum-Pemrograman-Mobile>